



## UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

---

# DOKUMENTASI BUKTI PEKERJAAN

## Kuis Refactor: Arsitektur Backend Terstruktur

*Route, Controller, Model, dan Middleware* pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

|                            |                                                                                                                               |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Nama</b>                | Delia Zahrani                                                                                                                 |
| <b>NPM</b>                 | 2428240111                                                                                                                    |
| <b>Kelas</b>               | SI5B                                                                                                                          |
| <b>Topik Tugas 1</b>       | Vaccinations (Topik 31: Puskesmas - Jadwal Vaksinasi)                                                                         |
| <b>Entitas utama</b>       | Vaccination (jadwal vaksinasi)                                                                                                |
| <b>Tautan repositori</b>   | <a href="https://github.com/deliazahrani444-cmd/si5c_kuis1_delia">https://github.com/deliazahrani444-cmd/si5c_kuis1_delia</a> |
| <b>Dosen pengampu</b>      | Nur Rachmat, M.Kom.                                                                                                           |
| <b>Tanggal pengumpulan</b> |                                                                                                                               |

## Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

### A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2\_Kuis2\_SI5C\_[NIM]\_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

### B. Syarat tangkapan layar

| Jenis                     | Yang harus terlihat                                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Terminal                  | Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.                                                                                                        |
| Kode                      | Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.                                                                                                                        |
| Pengujian <i>endpoint</i> | Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal. |
| Larangan                  | Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.                                           |

**Cara mengambil screenshot:** Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

### C. Susunan dokumen

| Bab   | Isi                                | Bukti utama                                    |
|-------|------------------------------------|------------------------------------------------|
| Bab 1 | Instalasi                          | 1 screenshot terminal                          |
| Bab 2 | Struktur folder dan ringkasan kode | 1 screenshot struktur folder, tabel penjelasan |
| Bab 3 | Menjalankan server                 | 1 screenshot terminal                          |

| Bab   | Isi                                      | Bukti utama          |
|-------|------------------------------------------|----------------------|
| Bab 4 | Pengujian endpoint (5 berhasil, 4 gagal) | 9 screenshot         |
| Bab 5 | Rekapitulasi hasil uji                   | Tabel rekap          |
| Bab 6 | Repositori kode                          | Tautan GitHub        |
| Bab 7 | Refleksi                                 | Dua jawaban tertulis |

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

## Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

### Data topik

|                                  |                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Topik Tugas 1</b>             | Vaccinations (Topik 31: Puskesmas - Jadwal Vaksinasi)                                                                               |
| <b>Entitas utama (tunggal)</b>   | Vaccination (jadwal vaksinasi)                                                                                                      |
| <b>Alamat dasar</b>              | /vaccinations                                                                                                                       |
| <b>Field data</b>                | id, namaPasien, nik, jenisVaksin, dosisKe, tanggal                                                                                  |
| <b>Field wajib saat POST</b>     | namaPasien, nik, jenisVaksin, dosisKe, tanggal (id dibuat otomatis oleh server)                                                     |
| <b>Query string untuk filter</b> | jenisVaksin, contoh: GET /vaccinations?jenisVaksin=Hepatitis B                                                                      |
| <b>Data awal</b>                 | 4 data: id 1 Rudi Santoso (Hepatitis B), id 2 Siti Aminah (Influenza), id 3 Budi Hartono (Hepatitis B), id 4 Maya Lestari (Tetanus) |
| <b>Modul lain (jika ada)</b>     | Tidak ada (hanya rute informasi GET /)                                                                                              |

### Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

| No | Metode | Alamat            | API key | Status sukses | Fungsi                                    |
|----|--------|-------------------|---------|---------------|-------------------------------------------|
| 1  | GET    | /vaccinations     | Tidak   | 200           | Ambil semua data (dengan filter bila ada) |
| 2  | GET    | /vaccinations/:id | Tidak   | 200           | Ambil satu data                           |
| 3  | POST   | /vaccinations     | Ya      | 201           | Tambah data                               |
| 4  | PUT    | /vaccinations/:id | Ya      | 200           | Ubah data                                 |
| 5  | DELETE | /vaccinations/:id | Ya      | 204           | Hapus data                                |

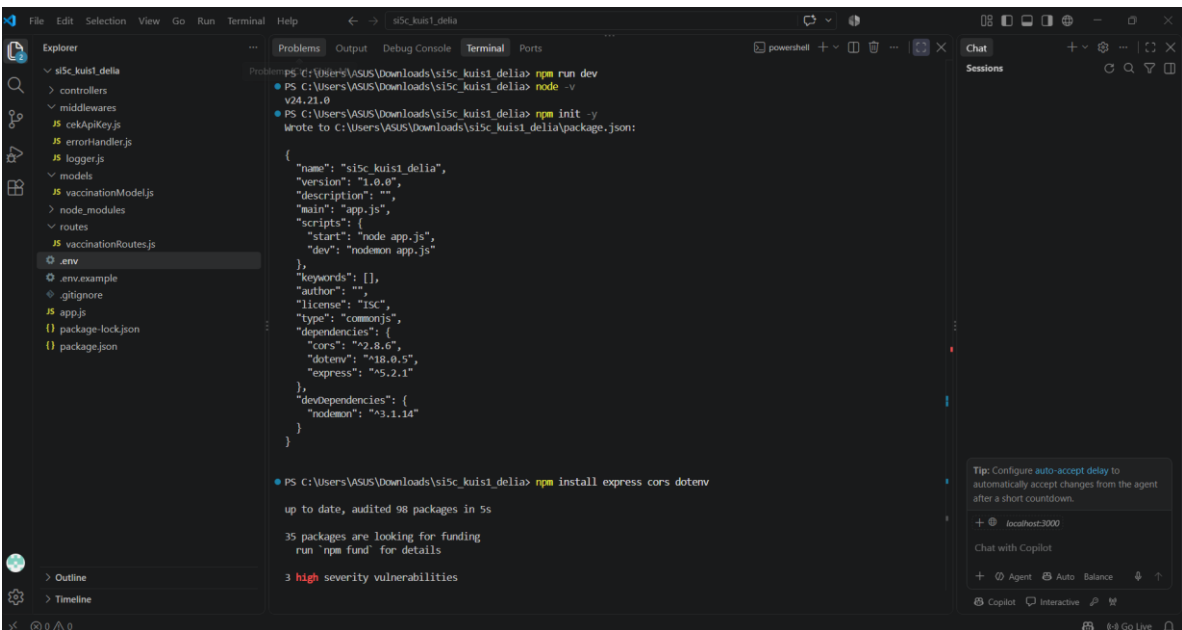
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

## Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

### 1.1 Instalasi project

**Harus terlihat pada screenshot:** perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```

PS C:\Users\ASUS\Downloads\si5c_kuis1_delia> npm run dev
PS C:\Users\ASUS\Downloads\si5c_kuis1_delia> node -v
v24.21.0
PS C:\Users\ASUS\Downloads\si5c_kuis1_delia> npm init -y
Wrote to C:\Users\ASUS\Downloads\si5c_kuis1_delia\package.json:

{
  "name": "si5c_kuis1_delia",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.5",
    "express": "^5.2.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}

PS C:\Users\ASUS\Downloads\si5c_kuis1_delia> npm install express cors dotenv

up to date, audited 98 packages in 5s
35 packages are looking for funding
run 'npm fund' for details
3 high severity vulnerabilities
  
```

Gambar 1. Versi Node.js dan instalasi paket

#### Penjelasan:

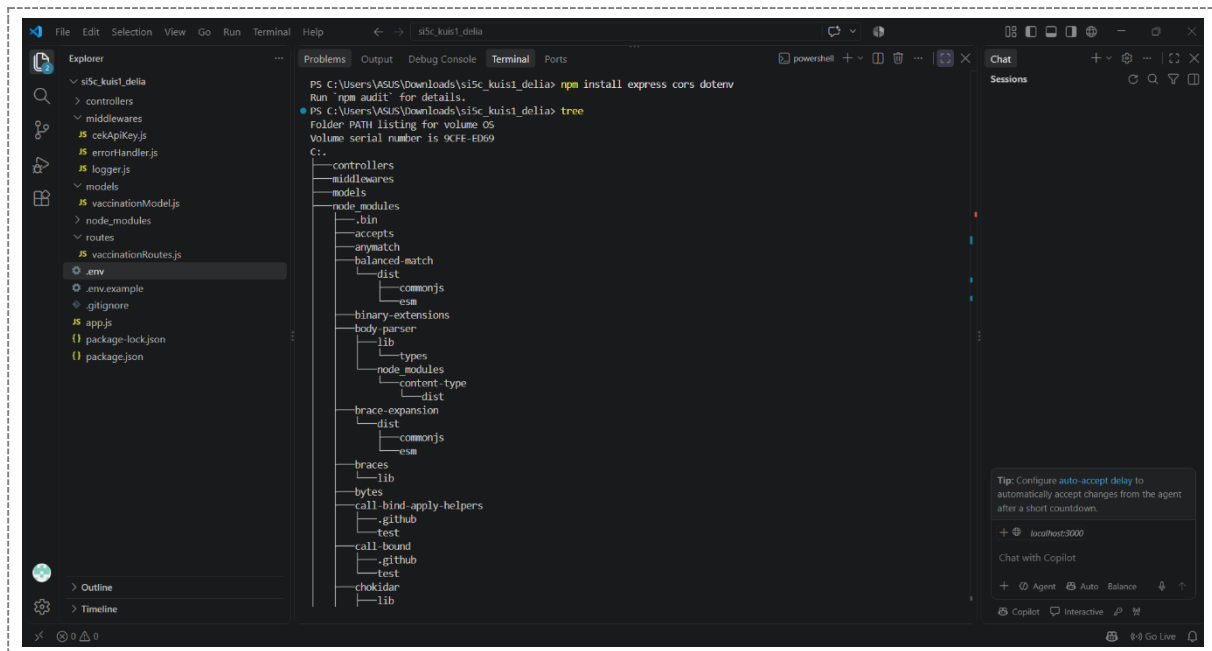
Express adalah framework web untuk Node.js yang saya pakai untuk membuat server dan mendaftarkan rute `/vaccinations`. `cors` mengizinkan API dipanggil dari alamat atau port lain (misalnya dari frontend) lewat `app.use(cors())`. `dotenv` membaca berkas `.env` sehingga `PORT` dan `API_KEY` tidak ditulis langsung di dalam kode. Express 5 membutuhkan Node.js versi baru (minimal Node 18), dan versi Node.js saya yang tampak pada Gambar 1 sudah memenuhi syarat itu.

## Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

### 2.1 Struktur folder

**Harus terlihat pada screenshot:** folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

#### Penjelasan:

routes memetakan alamat URL ke fungsi controller; controllers menerima request, memvalidasi data, dan mengirim response; models menyimpan array vaccinations beserta fungsi pengolahnya tanpa req dan res; middlewares berisi logger, cekApiKey, dan errorHandler yang berjalan di antara request dan controller.

### 2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

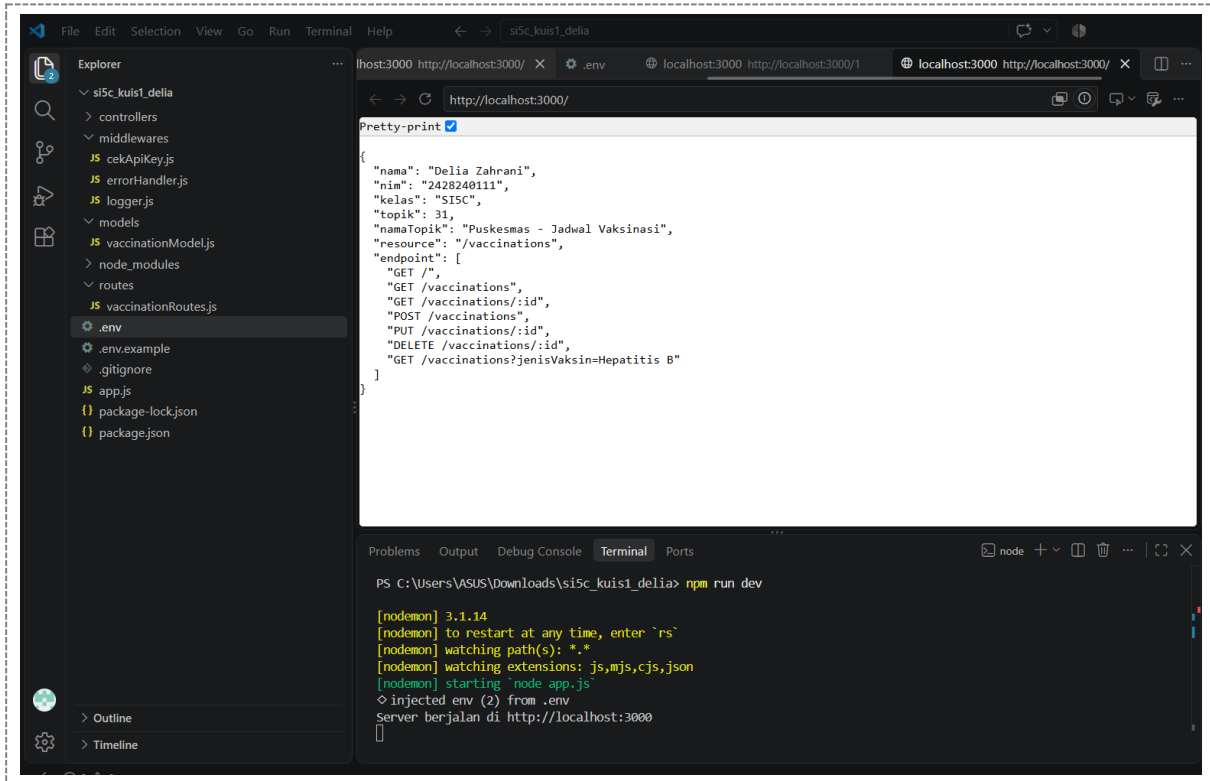
| Berkas                               | Penjelasan Anda                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| models/vaccinationModel.js           | Menyimpan array vaccinations (4 data awal) dan variabel nextId yang dimulai dari 5. Berisi fungsi getAll (dengan filter jenisVaksin), getByld, create, update, dan remove, semuanya tanpa memakai req atau res.                                                                                                                                          |
| controllers/vaccinationController.js | Berisi getAllVaccinations, getVaccinationByld, createVaccination, updateVaccination, dan deleteVaccination. Fungsi validasi memeriksa field wajib, dosisKe harus angka lebih dari 0, dan tanggal harus berformat YYYY-MM-DD; fungsi kirim membuat response berformat { status, message, data } dan controller memilih kode 200, 201, 204, 400, atau 404. |

| Berkas                      | Penjelasan Anda                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| routes/vaccinationRoutes.js | Membuat router dengan <code>express.Router()</code> dan memetakan GET /, GET /:id, POST /, PUT /:id, dan DELETE /:id ke fungsi di <code>vaccinationController</code> . Middleware <code>cekApiKey</code> dipasang hanya pada POST, PUT, dan DELETE.                                                                                                                             |
| middlewares/logger.js       | Mencetak waktu, metode, dan URL setiap request yang masuk ke terminal, lalu memanggil <code>next()</code> agar request diteruskan.                                                                                                                                                                                                                                              |
| middlewares/cekApiKey.js    | Membaca header <code>x-api-key</code> dan membandingkannya dengan <code>process.env.API_KEY</code> . Jika kosong atau berbeda, langsung membalas 401 tanpa memanggil <code>next()</code> ; jika cocok, request diteruskan ke controller.                                                                                                                                        |
| middlewares/errorHandler.js | Berisi <code>notFound</code> yang membalas 404 untuk rute yang tidak ada, dan <code>errorHandler</code> yang mengubah <code>error entity.parse.failed</code> (JSON rusak) menjadi 400 berformat JSON, sedangkan error lain menjadi 500.                                                                                                                                         |
| app.js                      | Memuat <code>dotenv</code> , membuat aplikasi Express, memasang <code>cors</code> , <code>express.json()</code> , dan <code>logger</code> , lalu mendaftarkan GET / dan <code>app.use("/vaccinations", vaccinationRoutes)</code> . <code>notFound</code> dan <code>errorHandler</code> dipasang paling akhir, dan <code>app.listen</code> memakai PORT dari <code>.env</code> . |

## Bab 3 Menjalankan Server

### 3.1 Menjalankan server

**Harus terlihat pada screenshot:** perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

#### Penjelasan:

Nomor port diambil dari `process.env.PORT`, yaitu nilai `PORT=3000` di berkas `.env` yang dibaca `dotenv`; jika `PORT` tidak ada, `app.js` memakai `3000` sebagai cadangan. Bila port itu sudah dipakai aplikasi lain, server gagal berjalan dengan error `EADDRINUSE`. Solusinya mengganti `PORT` di `.env` atau menghentikan aplikasi yang memakai port tersebut.

## Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

### Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

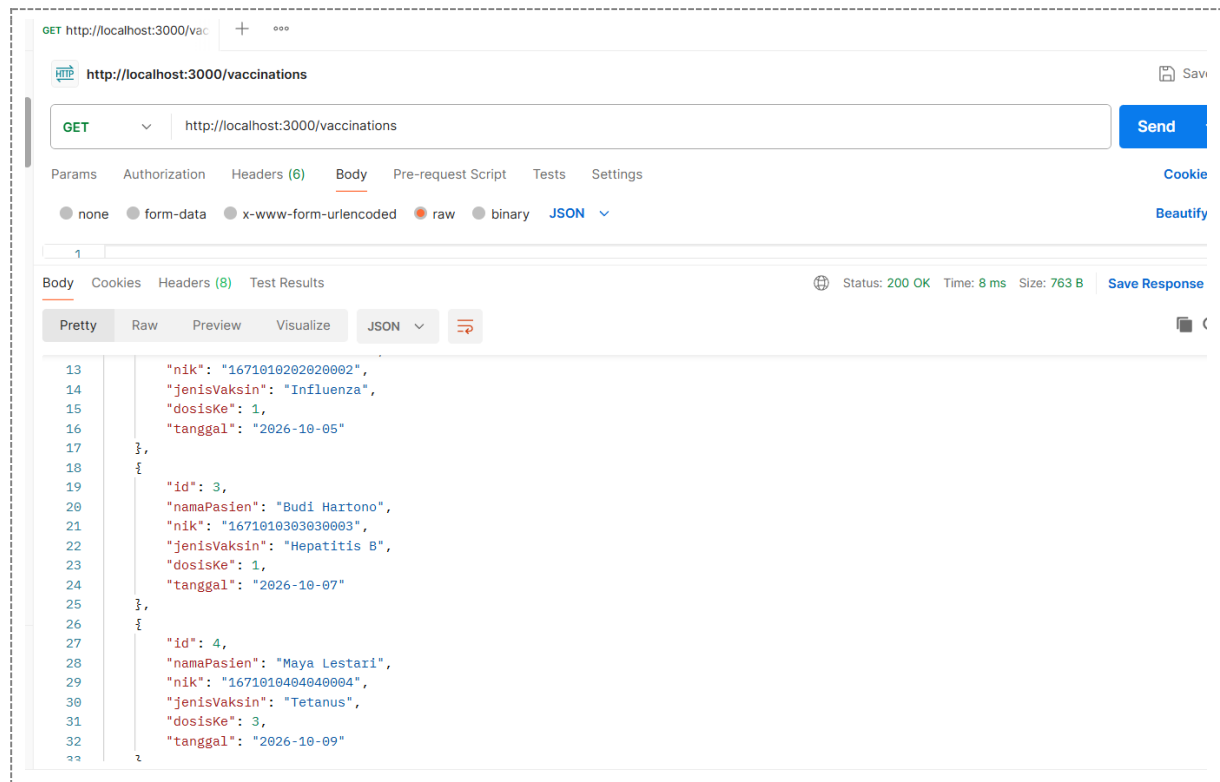
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

### Bagian A. Skenario berhasil

#### T01 Menampilkan semua data (GET /vaccinations)

**Harus terlihat pada screenshot:** metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

|                                    |                                                                       |
|------------------------------------|-----------------------------------------------------------------------|
| <b>Permintaan (<i>request</i>)</b> | GET /vaccinations                                                     |
| <b>Header</b>                      | (tidak ada)                                                           |
| <b>Body</b>                        | (tidak ada)                                                           |
| <b>Hasil yang diharapkan</b>       | 200 OK; isi berupa larik JSON seluruh data.                           |
| <b>Status hasil uji</b>            | 200 OK; larik JSON berisi 4 data vaksinasi (id 1 sampai 4)            |
| <b>Lulus?</b>                      | <input checked="" type="checkbox"/> Ya <input type="checkbox"/> Tidak |



Gambar 4. Hasil GET semua data

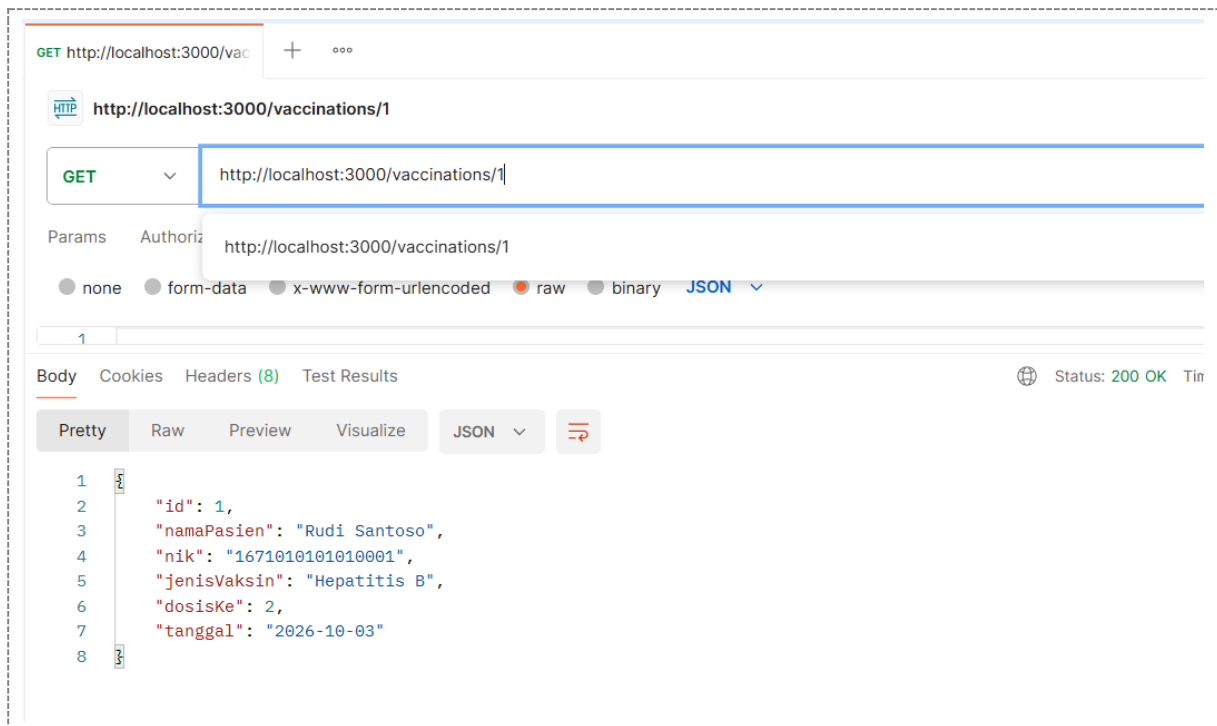
**Penjelasan:**

Status 200 OK berarti permintaan berhasil, dan responsnya larik JSON berisi seluruh data vaksinasi (bisa difilter dengan `?jenisVaksin=Hepatitis B`). Alurnya: `app.js` meneruskan `/vaccinations` ke `vaccinationRoutes`, route `router.get("/")` memanggil `getAllVaccinations` di controller, lalu controller memanggil `Vaccination.getAll()` di model yang mengembalikan array `vaccinations`.

**T02 Menampilkan satu data berdasarkan ID**

**Harus terlihat pada screenshot:** ID pada alamat dan objek tunggal pada respons.

|                              |                                                    |
|------------------------------|----------------------------------------------------|
| <b>Permintaan (request)</b>  | GET /vaccinations/1                                |
| <b>Header</b>                | (tidak ada)                                        |
| <b>Body</b>                  | (tidak ada)                                        |
| <b>Hasil yang diharapkan</b> | 200 OK; satu objek JSON dengan id yang diminta.    |
| <b>Status hasil uji</b>      | 200 OK; satu objek JSON dengan id 1 (Rudi Santoso) |
| <b>Lulus?</b>                | [ X ] Ya    [ ] Tidak                              |



Gambar 5. Hasil GET berdasarkan ID

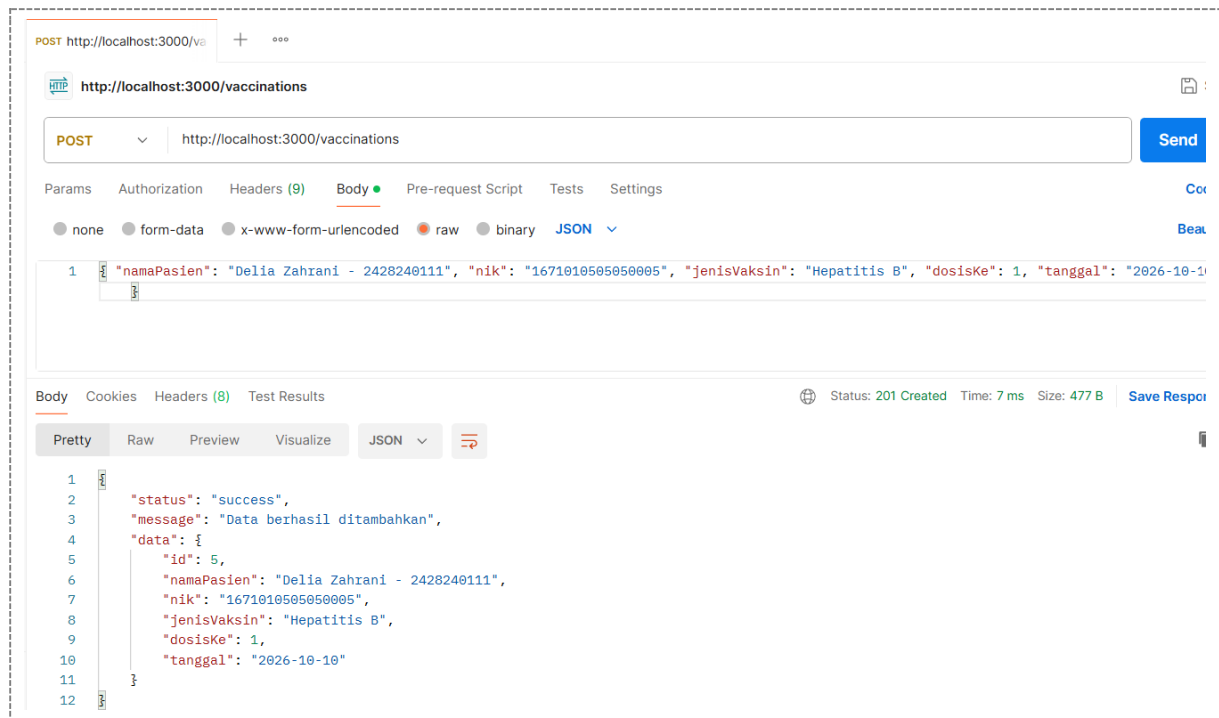
**Penjelasan:**

ID diambil dari alamat lewat `req.params.id`. Nilainya selalu berupa string, jadi saya ubah dengan `parseInt` agar bisa dibandingkan dengan `v.id` yang bertipe angka di `Vaccination.getById`. Tanpa konversi, pencarian dengan `===` tidak akan menemukan data.

**T03 Menambah data baru (POST)**

**Harus terlihat pada screenshot:** header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

|                              |                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Permintaan (request)</b>  | POST /vaccinations                                                                                                                             |
| <b>Header</b>                | x-api-key: vaksinasi123<br>Content-Type: application/json                                                                                      |
| <b>Body</b>                  | { "namaPasien": "Delia Zahrani - 2428240111", "nik": "1671010505050005", "jenisVaksin": "Hepatitis B", "dosisKe": 1, "tanggal": "2026-10-10" } |
| <b>Hasil yang diharapkan</b> | 201 Created; objek baru beserta id yang dibuat server.                                                                                         |
| <b>Status hasil uji</b>      | 201 Created; pesan "Data berhasil ditambahkan", objek baru dengan id 5                                                                         |
| <b>Lulus?</b>                | [ X ] Ya    [ ] Tidak                                                                                                                          |



Gambar 6. Hasil POST data baru

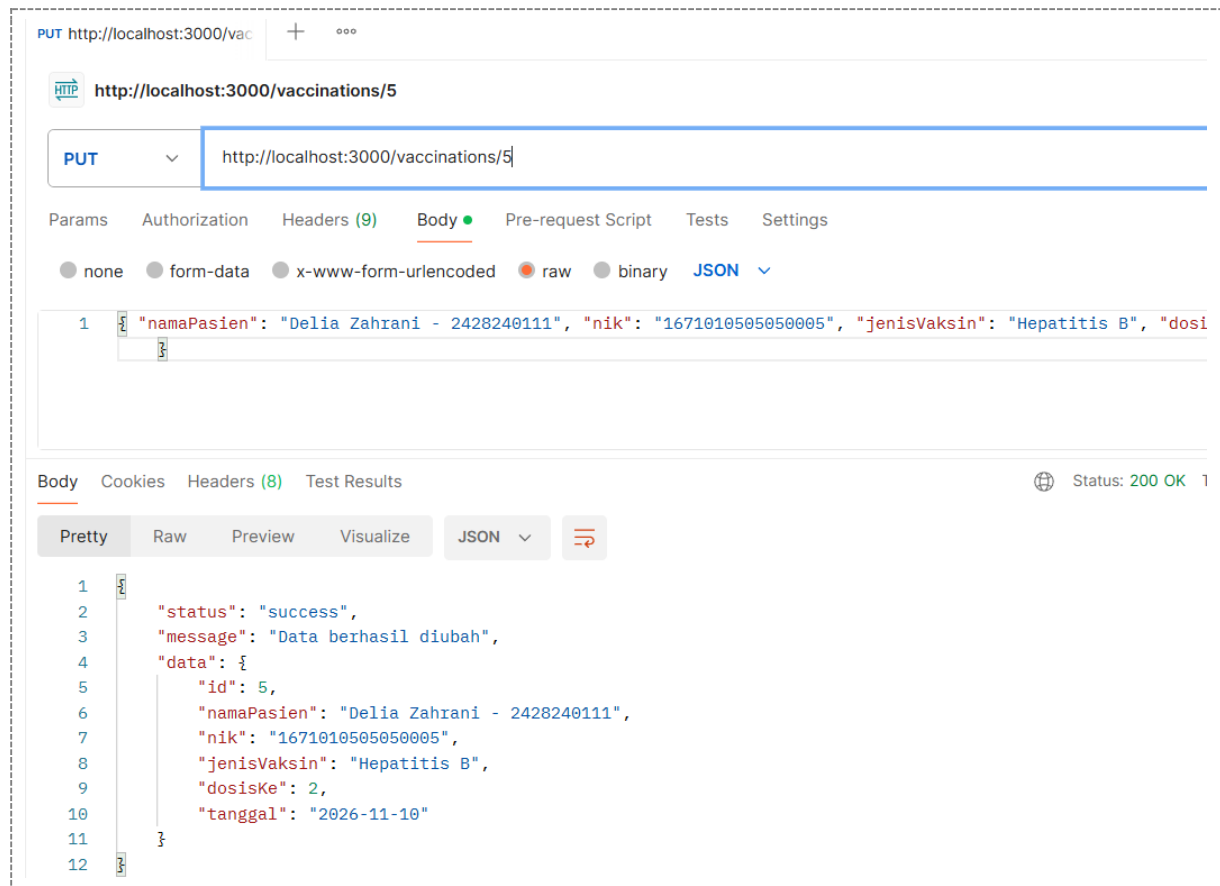
**Penjelasan:**

Header x-api-key diperiksa middleware cekApiKey yang dipasang pada route POST, lalu dibandingkan dengan API\_KEY di .env. express.json() mengubah body JSON menjadi objek req.body supaya bisa divalidasi oleh fungsi validasi dan disimpan oleh Vaccination.create. Statusnya 201 Created, bukan 200, karena permintaan ini membuat data baru (id 5) di server.

**T04 Mengubah data (PUT)**

**Harus terlihat pada screenshot:** ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

|                              |                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Permintaan (request)</b>  | PUT /vaccinations/5                                                                                                                            |
| <b>Header</b>                | x-api-key: vaksinasi123<br>Content-Type: application/json                                                                                      |
| <b>Body</b>                  | { "namaPasien": "Delia Zahrani - 2428240111", "nik": "1671010505050005", "jenisVaksin": "Hepatitis B", "dosisKe": 2, "tanggal": "2026-11-10" } |
| <b>Hasil yang diharapkan</b> | 200 OK; objek dengan field yang sudah berubah.                                                                                                 |
| <b>Status hasil uji</b>      | 200 OK; pesan "Data berhasil diubah", dosisKe menjadi 2 dan tanggal menjadi 2026-11-10                                                         |
| <b>Lulus?</b>                | [ X ] Ya    [ ] Tidak                                                                                                                          |



Gambar 7. Hasil PUT mengubah data

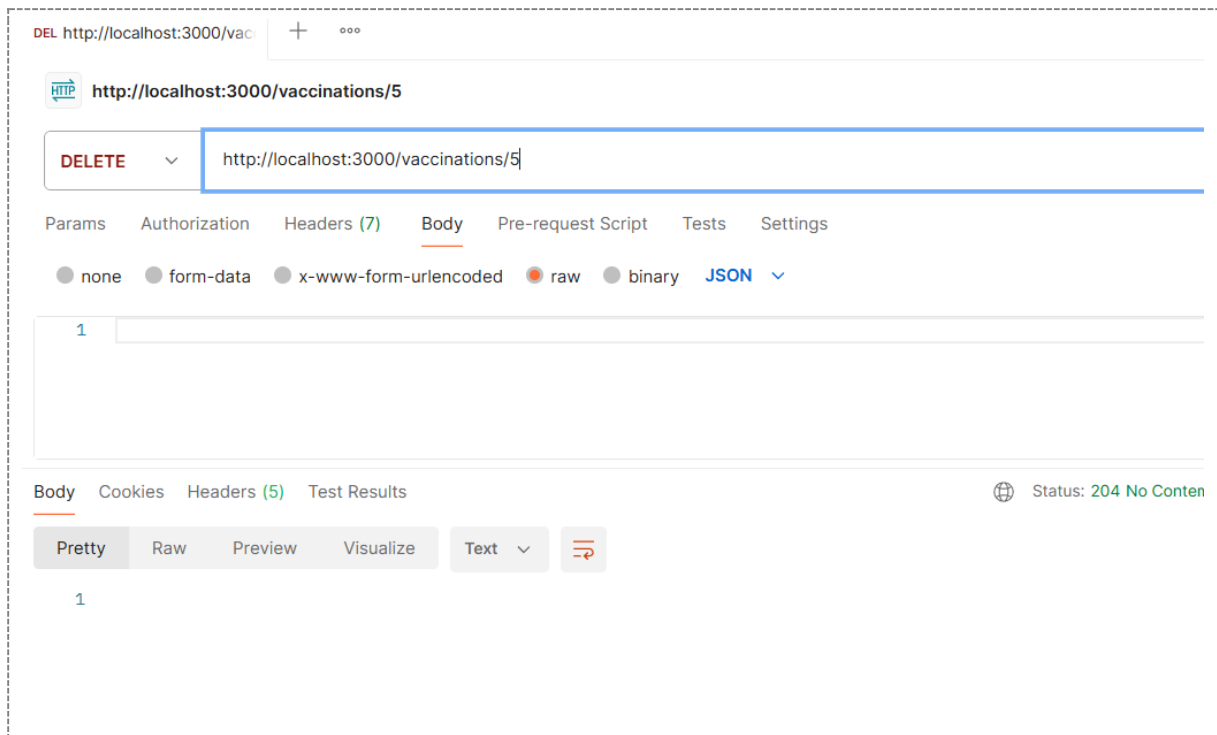
**Penjelasan:**

Yang berubah adalah dosisKe (dari 1 menjadi 2) dan tanggal (menjadi 2026-11-10); id, namaPasien, nik, dan jenisVaksin tetap sama. Karena PUT adalah penggantian penuh, semua field wajib saya kirim lengkap. Controller hanya memvalidasi dan membalas, sedangkan penggantian datanya dilakukan di layer Model oleh Vaccination.update.

**T05 Menghapus data (DELETE)**

**Harus terlihat pada screenshot:** metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

|                              |                                    |
|------------------------------|------------------------------------|
| <b>Permintaan (request)</b>  | DELETE /vaccinations/5             |
| <b>Header</b>                | x-api-key: vaksinasi123            |
| <b>Body</b>                  | (tidak ada)                        |
| <b>Hasil yang diharapkan</b> | 204 No Content; tanpa isi respons. |
| <b>Status hasil uji</b>      | 204 No Content; tanpa isi respons  |
| <b>Lulus?</b>                | [ X ] Ya   [ ] Tidak               |



Gambar 8. Hasil DELETE data

**Penjelasan:**

Status 204 No Content berarti permintaan berhasil dan data dengan id 5 sudah dihapus oleh Vaccination.remove. Respons sengaja tidak berisi apa pun karena tidak ada lagi data yang perlu dikirim kembali ke klien.

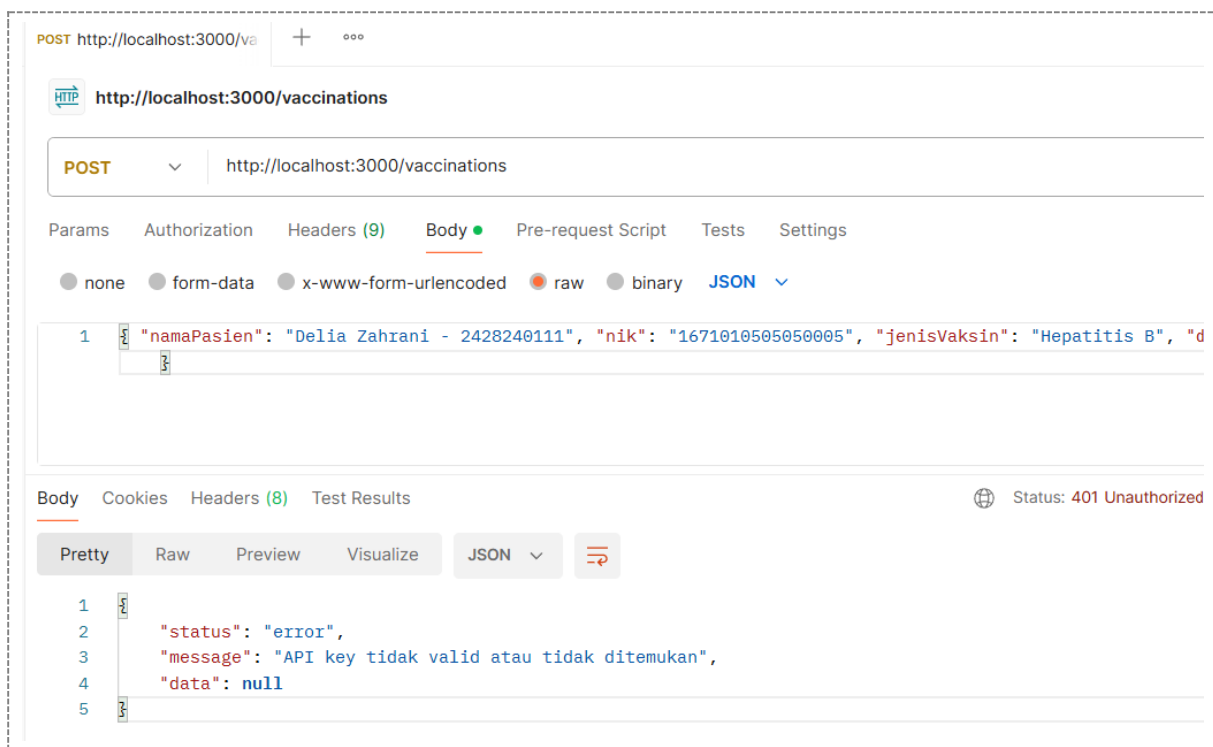
## Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

### N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

|                       |                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Permintaan (request)  | POST /vaccinations (tanpa x-api-key)                                                                                                           |
| Header                | Content-Type: application/json                                                                                                                 |
| Body                  | { "namaPasien": "Delia Zahrani - 2428240111", "nik": "1671010505050005", "jenisVaksin": "Hepatitis B", "dosisKe": 1, "tanggal": "2026-10-10" } |
| Hasil yang diharapkan | 401 Unauthorized; pesan JSON; data tidak bertambah.                                                                                            |
| Status hasil uji      | 401 Unauthorized; pesan "API key tidak valid atau tidak ditemukan"                                                                             |
| Lulus?                | [ X ] Ya [ ] Tidak                                                                                                                             |



Gambar 9. POST tanpa API key ditolak

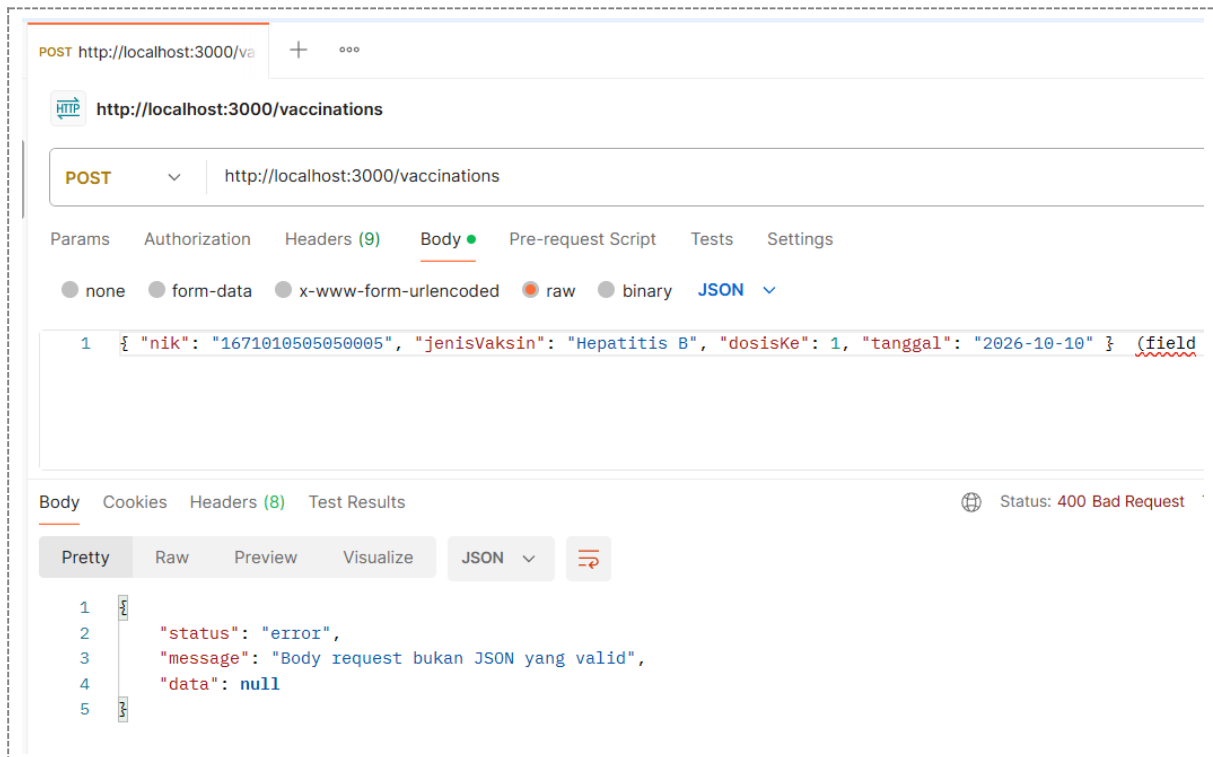
#### Penjelasan:

Penolakannya adalah middleware cekApiKey. Karena header x-api-key tidak ada, middleware langsung membalas 401 dan tidak memanggil next(), sehingga createVaccination di controller tidak pernah dijalankan dan data tidak bertambah.

## N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

|                              |                                                                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Permintaan (request)</b>  | POST /vaccinations                                                                                                                        |
| <b>Header</b>                | x-api-key: vaksinasi123<br>Content-Type: application/json                                                                                 |
| <b>Body</b>                  | { "nik": "1671010505050005", "jenisVaksin": "Hepatitis B", "dosisKe": 1, "tanggal": "2026-10-10" } (field namaPasien sengaja dihilangkan) |
| <b>Hasil yang diharapkan</b> | 400 Bad Request; pesan menyebut field yang wajib.                                                                                         |
| <b>Status hasil uji</b>      | 400 Bad Request; pesan "Field namaPasien wajib diisi"                                                                                     |
| <b>Lulus?</b>                | [ X ] Ya    [ ] Tidak                                                                                                                     |



Gambar 10. POST dengan body tidak lengkap

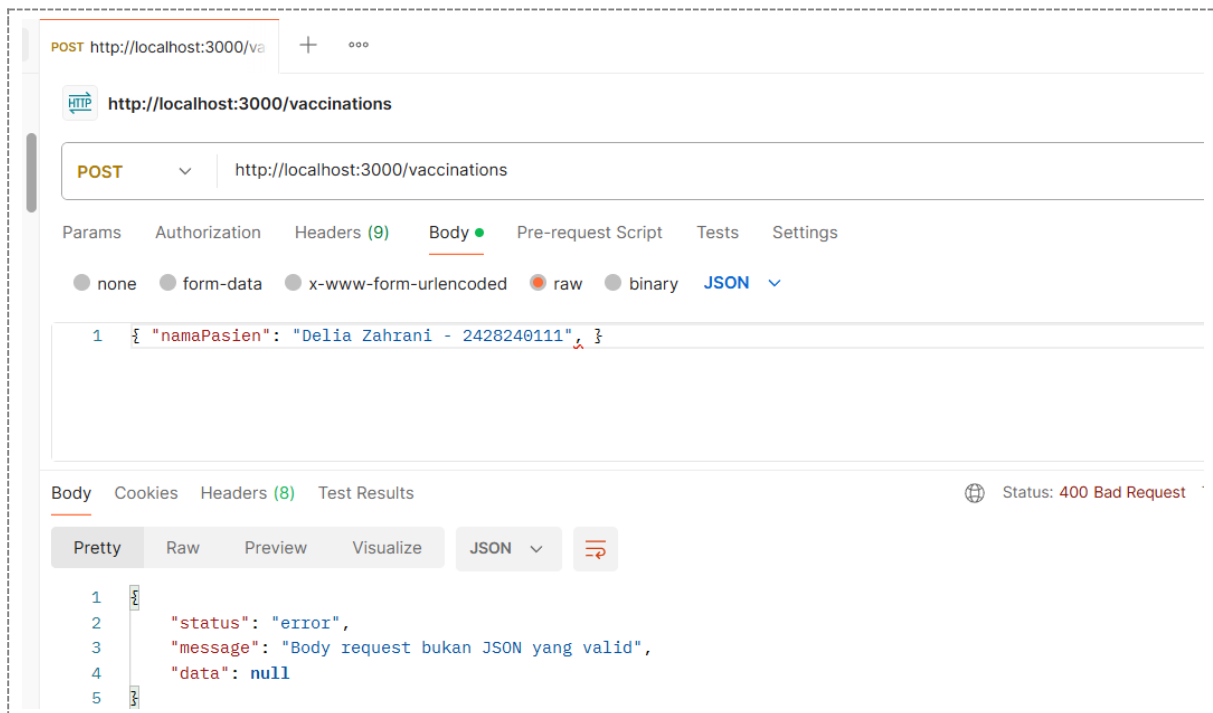
### Penjelasan:

Validasi ditulis di layer controller, yaitu fungsi validasi yang dipanggil `createVaccination` sebelum `Vaccination.create`. Statusnya 400 Bad Request karena kesalahannya ada pada data kiriman klien (field `namaPasien` tidak diisi), bukan pada server.

### N03 Mengirim JSON yang rusak

**Harus terlihat pada screenshot:** body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

|                              |                                                                        |
|------------------------------|------------------------------------------------------------------------|
| <b>Permintaan (request)</b>  | POST /vaccinations                                                     |
| <b>Header</b>                | x-api-key: vaksinasi123<br>Content-Type: application/json              |
| <b>Body</b>                  | { "namaPasien": "Delia Zahrani - 2428240111", } (koma berlebih)        |
| <b>Hasil yang diharapkan</b> | 400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML). |
| <b>Status hasil uji</b>      | 400 Bad Request; pesan "Body request bukan JSON yang valid"            |
| <b>Lulus?</b>                | [ X ] Ya   [ ] Tidak                                                   |



Gambar 11. POST dengan JSON rusak

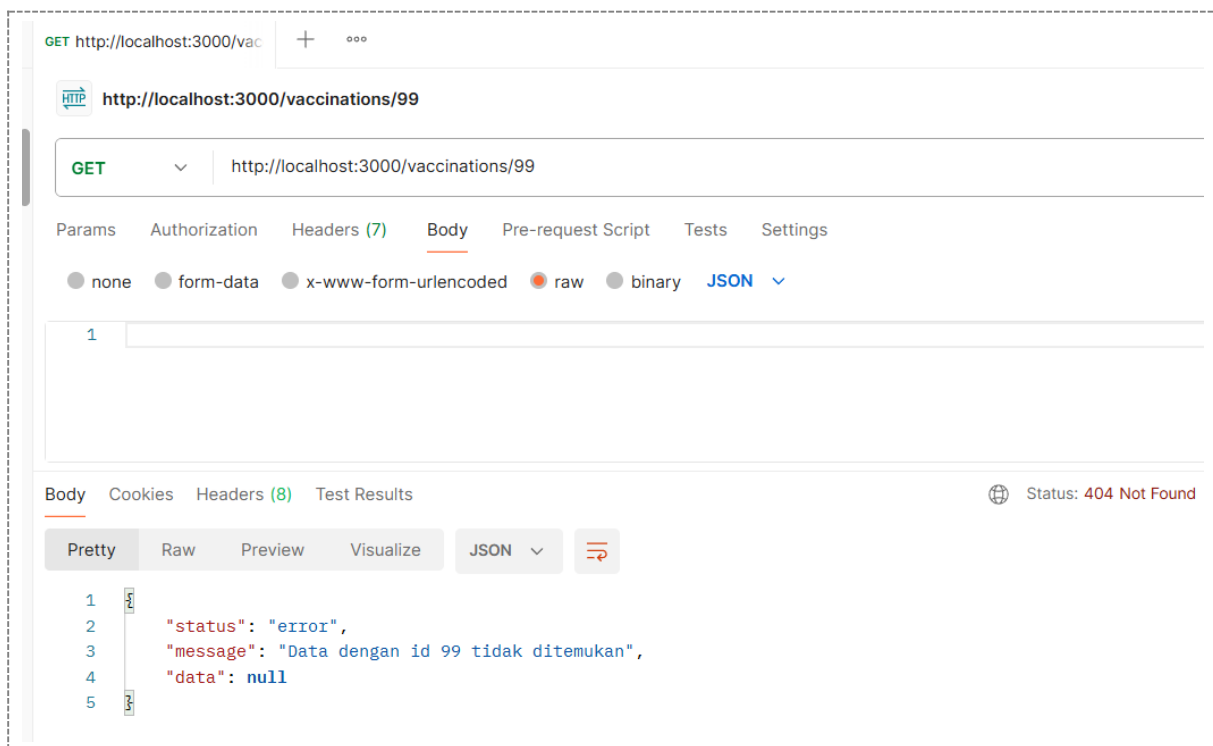
#### Penjelasan:

JSON rusak dideteksi oleh `express.json()` saat mem-parsing body, yang melempar error bertipe `entity.parse.failed` sebelum request sampai ke controller. Error itu diteruskan ke errorHandler terpusat yang mengubahnya menjadi respons JSON 400 "Body request bukan JSON yang valid", bukan halaman HTML berisi jejak error.

## N04 Mengambil ID yang tidak ada

**Harus terlihat pada screenshot:** ID yang jelas tidak ada (misalnya 99) dan respons 404.

|                              |                                                          |
|------------------------------|----------------------------------------------------------|
| <b>Permintaan (request)</b>  | GET /vaccinations/99                                     |
| <b>Header</b>                | (tidak ada)                                              |
| <b>Body</b>                  | (tidak ada)                                              |
| <b>Hasil yang diharapkan</b> | 404 Not Found; pesan JSON.                               |
| <b>Status hasil uji</b>      | 404 Not Found; pesan "Data dengan id 99 tidak ditemukan" |
| <b>Lulus?</b>                | [ X ] Ya    [ ] Tidak                                    |



Gambar 12. GET ID tidak ada

### Penjelasan:

Request GET /vaccinations/99 melewati logger, lalu route `router.get('/:id')` memanggil `getVaccinationById`. Controller memanggil `Vaccination.getById(99)` yang mengembalikan `undefined`, kemudian fungsi `tidakDitemukan` di controller mengirim respons 404 berisi pesan "Data dengan id 99 tidak ditemukan". Jadi yang mengirim pesan 404 ini adalah controller.

## Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

| ID  | Skenario                                | Status harapan | Status hasil | Lulus |
|-----|-----------------------------------------|----------------|--------------|-------|
| T01 | Menampilkan semua data                  | 200            | 200          | Ya    |
| T02 | Menampilkan satu data berdasarkan ID    | 200            | 200          | Ya    |
| T03 | Menambah data baru                      | 201            | 201          | Ya    |
| T04 | Mengubah data                           | 200            | 200          | Ya    |
| T05 | Menghapus data                          | 204            | 204          | Ya    |
| N01 | Menambah data tanpa API key             | 401            | 401          | Ya    |
| N02 | Menambah data dengan body tidak lengkap | 400            | 400          | Ya    |
| N03 | Mengirim JSON yang rusak                | 400            | 400          | Ya    |
| N04 | Mengambil ID yang tidak ada             | 404            | 404          | Ya    |

## Bab 6 Repositori Kode

---

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

|                                                            |                                                                                                                               |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Tautan repositori</b>                                   | <a href="https://github.com/deliazahrani444-cmd/si5c_kuis1_delia">https://github.com/deliazahrani444-cmd/si5c_kuis1_delia</a> |
| <b>Pesan commit terakhir</b>                               | Refactor API vaksinasi ke arsitektur terstruktur                                                                              |
| <b>Folder routes, controllers, models, middlewares ada</b> | <input checked="" type="checkbox"/> Ya <input type="checkbox"/> Tidak                                                         |
| <b>.env tidak terunggah (ada di .gitignore)</b>            | <input checked="" type="checkbox"/> Ya <input type="checkbox"/> Tidak                                                         |
| <b>.env.example ada</b>                                    | <input checked="" type="checkbox"/> Ya <input type="checkbox"/> Tidak                                                         |

## Bab 7 Refleksi

---

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

### Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

**Penjelasan:**

Route hanya memetakan alamat ke fungsi, contohnya `router.post("/", cekApiKey, c.createVaccination)` di `vaccinationRoutes.js`. Controller menangani request, validasi, dan response, contohnya  `kirim(res, 201, "success", "Data berhasil ditambahkan", baru)` di `createVaccination`. Model hanya mengolah data tanpa req dan res, contohnya `vaccinations.push(baru)` di `Vaccination.create`.

### Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

**Penjelasan:**

Dengan error handler terpusat, format respons error cukup ditulis sekali di `errorHandler.js` sehingga route dan controller tidak perlu mengulang penanganan error. Buktinya pada N03: tidak ada satu pun route atau controller yang menangani JSON rusak, tetapi API tetap membalas 400 berformat JSON yang rapi karena error ditangkap `errorHandler`. Route yang tidak ada juga otomatis menghasilkan 404 JSON dari `notFound`.

## Lampiran A Daftar Periksa Sebelum Mengumpulkan

| No | Butir yang diperiksa                                                  | Ya |
|----|-----------------------------------------------------------------------|----|
| 1  | Seluruh tanda [ . . . ] sudah diganti dengan topik Tugas 1 saya       |    |
| 2  | Semua tangkapan layar berasal dari komputer saya dan terbaca jelas    |    |
| 3  | Setiap gambar memiliki nomor, keterangan, dan penjelasan              |    |
| 4  | Identitas saya (nama atau NIM) muncul pada data uji POST              |    |
| 5  | Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji |    |
| 6  | Tabel rekapitulasi sesuai dengan tangkapan layar                      |    |
| 7  | Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal   |    |
| 8  | Berkas disimpan sebagai PDF dengan nama sesuai ketentuan              |    |

## Lampiran B Rubrik Penilaian (Usulan)

*Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.*

| Komponen                                            | Kriteria penilaian penuh                                                                          | Bobot      |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------|------------|
| Instalasi dan struktur folder (Bab 1 sampai 3)      | Instalasi, struktur folder, dan server berjalan benar                                             | 15         |
| Ringkasan kode dan kode di repositori (Bab 2 dan 6) | Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal | 30         |
| Pengujian skenario berhasil (T01 sampai T05)        | Status dan respons sesuai; penjelasan benar                                                       | 25         |
| Pengujian skenario gagal (N01 sampai N04)           | Kode status dan pesan JSON sesuai; penjelasan alur error benar                                    | 20         |
| Rekapitulasi dan refleksi (Bab 5 dan 7)             | Rekap konsisten dengan bukti; jawaban pribadi                                                     | 10         |
| <b>Total</b>                                        |                                                                                                   | <b>100</b> |